
Quantum Key Distribution

- BB84 -

Project Report
QKD

Aalborg University
The Department of Computer Science



The Department of Computer Science
Aalborg University
<http://www.aau.dk>

AALBORG UNIVERSITY

STUDENT REPORT

Title:

Quantum Key Distribution

Theme:

Quantum Information and Computing
(QIC)

Project Period:

Spring Semester 2026

Project Group:

QKD

Participant(s):

Daniel Maxim:

dmaxim25@student.aau.dk

Mahsa Hemmatpanah:

mhemma25@student.aau.dk

Page Numbers: 14**Date of Completion:**

8

Abstract:

This project implements a basic BB84 Quantum Key Distribution protocol using Qiskit. The aim is to validate the fundamental principles of quantum-secure communication, namely the generation of a shared symmetric key and the detection of adversarial interference through quantum measurement disturbance and Quantum Bit Error Rate (QBER) analysis. To evaluate the robustness of the protocol under practical constraints, the implementation is executed across three environments: a noiseless theoretical baseline, a parameterized noisy simulator, and an IBM Quantum hardware. Temporal circuit batching is employed to overcome hardware limitations, enabling the separation of intrinsic device noise from disturbances caused by intercept-resend attacks.

Contents

1	Introduction	1
2	State of the art	1
3	method	2
3.1	Problem Statement: BB84 Key Distribution	2
3.2	Case 1: No Eve	3
3.3	Case 2: Passive Classical Eavesdropping	4
3.4	Case 3: Active Classical Eavesdropping	4
3.5	Feasibility Discussion	4
3.5.1	Can Bob Detect Eve Before Comparing Bases?	4
3.5.2	Can Eve's Disturbance Accidentally Disappear?	5
3.5.3	Why the Key Must Be Large Enough	5
3.5.4	Final Interpretation	5
3.5.5	Error Correction and Privacy Amplification	6
4	Implementation and Results	6
4.1	High-Level Description	6
4.2	Source Code Explanation	7
4.2.1	Quantum State Preparation	7
4.2.2	Classical Post-Processing	8
4.2.3	Execution Architecture	8
4.3	Execution Strategy	9
4.3.1	Testing Framework and Results	9
5	Discussion and Conclusion	10
5.1	Discussion of Simulation and Hardware Results	10
5.1.1	Ideal Simulation	10
5.1.2	Noisy Simulation	10
5.1.3	Hardware Execution and Environmental Comparison	11
5.1.4	Statistical Convergence	11
5.1.5	Limitations	12
5.2	Conclusion	12

Abbreviations

QIC	Quantum Information and Computing
QKD	Quantum Key Distribution
QBER	Quantum Bit Error Rate
BB84	Bennett and Brassard 1984 Protocol
MCX	Multi-Control X Gate
QC	Quantum Computing
Qubit	Quantum Bit
CX	Controlled-X Gate (CNOT)
CNOT	Controlled NOT Gate
X Gate	Pauli-X (Bit-flip) Gate
Z Gate	Pauli-Z (Phase-flip) Gate
H Gate	Hadamard Gate
QEC	Quantum Error Correction
I/O	Input / Output

Chapter 1

Introduction

Quantum Key Distribution (QKD) is a method for generating a shared secret key between two parties by exploiting the fundamental principles of quantum mechanics [1]. In the BB84 protocol, information is encoded into quantum states prepared in different measurement bases, so measuring a qubit in the wrong basis can change the state and introduce detectable errors[2].

This property is related to quantum superposition [3] and the No-Cloning Theorem, which states that an unknown quantum state cannot be copied perfectly [4]. Together, these constraints ensure that any attempt to observe or intercept quantum information leaves a measurable physical trace in the form of transmission errors [2, 1].

The problem addressed in this project is the implementation of a basic QKD protocol in order to study how quantum mechanics can support secure communication. To achieve this, the protocol is tested across multiple execution environments, including a noiseless simulator, a parameterized noisy simulator, and physical IBM Quantum processors.

Furthermore, an intercept-resend attack is introduced to evaluate how interference affects the Quantum Bit Error Rate (QBER). By analysing these results across different environments, the project investigates statistical behaviour, scalability, and the practical viability of quantum-secure communication on existing hardware platforms.

The implemented solution is publicly available at: <https://github.com/mahsa-hemmat/qiskit-bb84-hardware.git>

Chapter 2

State of the art

Quantum Key Distribution (QKD) has evolved from a theoretical physics concept into an active field of telecommunications engineering [1, 5]. While early research and literature established the physical principles and mathematical security proofs of foundational protocols like BB84, modern research is driven by the challenge of moving these protocols out of the laboratory and into the real world.

A major focus in current research is the integration of quantum hardware with existing classical network infrastructure. This involves researching optical-fibre transmission,

key management protocols, and scalable network simulations [6]. Furthermore, organizations like ETSI have developed standardized frameworks that define exactly how QKD-generated keys can be securely requested and delivered to software applications using standard REST-based protocols [7, 8].

Alongside networking challenges, researchers must address physical hardware limitations. Real-world quantum channels suffer from ambient noise, photon loss, and imperfect detectors. Because these physical imperfections threaten the mathematical security of the protocol, advanced variations of BB84 have been developed. For example, modern implementations utilise decoy states to mitigate photon-number-splitting (PNS) attacks and improve key generation rates over long distances [9, 10].

The scope of this project focuses exclusively on the algorithmic and cryptographic core of QKD. By building a software-based computational implementation of the BB84 protocol, this research abstracts away the physical optical hardware to directly observe the underlying quantum mechanics [2].

The primary objective is to demonstrate how basis sifting operates and to mathematically evaluate the protocol's disturbance-based security mechanism. Through simulations in both ideal and noisy environments, the implementation illustrates how the presence of an eavesdropper introduces measurable disturbances in the quantum states, resulting in a detectable increase in the Quantum Bit Error Rate (QBER) [1].

Chapter 3

method

3.1 Problem Statement: BB84 Key Distribution

Alice and Bob require a mechanism to communicate securely over a potentially compromised environment. Their communication infrastructure consists of two distinct channels:

- a quantum channel, used to send qubits
- a public but authenticated classical channel, used to compare bases and error testing

Although the classical channel is publicly accessible and can be monitored by an eavesdropper (Eve), authentication ensures that Eve cannot modify transmitted messages or impersonate either party [1].

In BB84, Alice encodes classical bits into quantum states using two possible bases [2]:

$$Z = \{|0\rangle, |1\rangle\}$$

and

$$X = \{|+\rangle, |-\rangle\},$$

where

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

Alice generates a random sequence of classical bits and independently selects a random sequence of bases. Each bit is encoded into a corresponding quantum state and transmitted to Bob over the quantum channel, respectively:

$$0 \mapsto |+\rangle \quad 1 \mapsto |-\rangle$$

for diagonal, and :

$$0 \mapsto |0\rangle \quad 1 \mapsto |1\rangle$$

for rectilinear.

3.2 Case 1: No Eve

Alice sends a sequence of quantum states, for example:

$$|+\rangle, |0\rangle, \dots, |-\rangle.$$

Bob measures each incoming qubit using his own independently generated sequence of random bases. After transmission, Alice and Bob publicly compare only their bases, not their bit values [2, 1]. They keep the qubits where Bob used the correct basis and discard the rest.

To verify channel integrity, they reveal a random subset of the sifted key. Since there is no Eve and no noise, all tested bits match:

Alice: 00100110100010011100

Bob: 00100110100010011100

The error rate is

$$QBER = 0.$$

The revealed bits are discarded and they only keep the remaining bits as the raw secret key. After error correction and privacy amplification, Alice and Bob obtain a shared secret key [1]. This key can be used for encryption, for example with a one-time pad:

$$C = M \oplus K,$$

where M is the message, K is the key, and C is the ciphertext. Bob decrypts with

$$M = C \oplus K.$$

3.3 Case 2: Passive Classical Eavesdropping

Suppose Eve does not touch the quantum channel and only listens to the public discussion where Alice and Bob compare bases.

This is not enough for Eve to learn the secret key, because Alice and Bob reveal only their bases, not the secret bit values [2, 1]. They may also reveal some test bits, but these are discarded afterward, so they are not part of the final key. Therefore, if Eve only listens after the quantum transmission, the protocol remains secure. However, the security of this phase relies on the authentication of the classical channel, which prevents message modification and man-in-the-middle attacks [1].

3.4 Case 3: Active Classical Eavesdropping

Now suppose Eve intercepts the qubits before Bob receives them. In this scenario, Eve measures the transmitted qubit and forwards a newly prepared qubit to Bob based on her measurement [2, 1]. Since Eve does not know Alice's basis, she must guess randomly and she may choose the wrong one. For example, Alice sends qubits in the diagonal basis X , but Eve measures some of them in the rectilinear basis Z . Measuring $|+\rangle$ in the wrong basis gives a random result:

$$|+\rangle \xrightarrow{\text{measure in } Z} \begin{cases} |0\rangle, & \text{with probability } \frac{1}{2}, \\ |1\rangle, & \text{with probability } \frac{1}{2}. \end{cases}$$

This disturbs the state before it reaches Bob. After Alice and Bob compare bases, they keep the matching-basis positions and reveal a fraction of the bits for error estimation. In the example below 25% of the bits are wrong since 5 bits out of 20 are mismatched.

Alice: 00100110100010011100

Bob: 1010001010001111101

$$QBER = \frac{5}{20} = 25\%.$$

Since this error rate exceeds acceptable security thresholds, Alice and Bob detect the presence of an eavesdropper and abort the protocol, discarding the compromised key.

3.5 Feasibility Discussion

3.5.1 Can Bob Detect Eve Before Comparing Bases?

In general, Bob cannot detect Eve from his own results alone [1]. During transmission, he does not know Alice's original bits or bases, so his measurement outcomes appear random regardless of Eve's presence. Therefore, detection becomes possible only when Alice and Bob compare subsets of their sifted key [2, 1].

3.5.2 Can Eve's Disturbance Accidentally Disappear?

Yes. Eve may disturb a qubit, but Bob can still obtain the correct bit by chance. Therefore, Eve's attack does not always create visible errors; it creates errors statistically [1, 5]. In an intercept-resend attack, Eve chooses the wrong basis with probability 50%, which leads to a 50% chance of error in Bob's measurement. This results in an expected QBER of approximately 25% in the sifted key [1, 5]. If Alice and Bob test only a few bits, Eve could get lucky and avoid detection. If the actual error rate is e , and they test t bits, the probability of observing no errors is approximately

$$(1 - e)^t.$$

For example, $e = 25\%$ and $t = 10$:

$$(0.75)^{10} \approx 0.056.$$

So there is still about a 5.6% chance that Eve remains undetected.

However, increasing the sample size to $t = 100$ significantly reduces this risk:

$$(0.75)^{100} \approx 3.2 \times 10^{-13}.$$

Thus, sufficiently large sample sizes ensure that Eve is detected with very high probability.

3.5.3 Why the Key Must Be Large Enough

The key must be large because Alice and Bob sacrifice some bits for testing, error correction, and privacy amplification [1]. The tested bits are revealed publicly and discarded, while the remaining bits are kept only if the observed error rate is low enough.

In practice, many more than 100 qubits are used, making it statistically unlikely that Eve can disturb the channel without being detected.

3.5.4 Final Interpretation

BB84 does not prevent eavesdropping. It makes eavesdropping detectable [2, 1]. The fundamental principle is that any attempt to gain information about an unknown quantum state inevitably introduces disturbance.

If Eve does not touch the transmitted qubits, she learns no useful information about the key. If she attempts to measure them, she introduces errors. Alice and Bob estimate this disturbance by computing the Quantum Bit Error Rate (QBER) on a subset of their sifted key. If the observed error rate exceeds a predefined security threshold, they abort the protocol. If the error rate is low, they use error correction and privacy amplification to obtain a shared secret key.

Therefore, BB84 is secure because Eve cannot learn quantum information without creating a disturbance that Alice and Bob can detect statistically [1].

3.5.5 Error Correction and Privacy Amplification

Even without Eve, Alice and Bob's sifted keys may differ because of channel noise and imperfections in quantum state preparation and measurement. Error correction lets them identify and correct mismatched bits through limited communication over the authenticated classical channel [1].

However, this communication may reveal some information about the key to Eve. To mitigate this, Alice and Bob perform privacy amplification. Privacy amplification reduces this risk by compressing the corrected key into a shorter final key, usually with a hash function. The final key is shorter, but more secure [1].

Chapter 4

Implementation and Results

4.1 High-Level Description

This Chapter explains the implementation of the BB84 Quantum Key Distribution (QKD) protocol using Qiskit. Key components, including circuit transpilation, hardware access, and noise modelling, were implemented in accordance with the official Qiskit and IBM Quantum documentation [11, 12]. The program accepts an input parameter n , representing the number of qubits to be transmitted, alongside configuration options for the execution environment (local simulator, noisy simulator, or real IBM Quantum hardware) and an optional intercept-resend attack flag (`use_eve`).

During the pre-processing stage, random bit sequences and measurement bases (rectilinear Z or diagonal X) are generated for both Alice and Bob. A quantum circuit is then constructed, utilizing Pauli-X and Hadamard gates to encode Alice's classical bits into quantum states. If the eavesdropping flag is enabled, mid-circuit measurements and re-preparation steps are inserted to simulate wave-function collapse, reducing the quantum state to a definite value upon interception.

To handle hardware constraints while preserving cryptographic security, execution is divided into two approaches. Local simulations leverage `backend.run(memory=True)` execution to preserve the chronological measurement order while safely injecting depolarising and readout noise models. Also, physical execution on real IBM Quantum hardware utilises a circuit-batching strategy, submitting multiple independent, randomised circuits through the Qiskit Runtime `SamplerV2` primitive.

After quantum execution, Alice and Bob publicly compare their measurement bases to sift the key. A random subset of this sifted key is sacrificed to compute the Quantum Bit Error Rate (QBER). The final output provides a structured diagnostic summary evaluating the

channel's security against the theoretical abort threshold.

4.2 Source Code Explanation

The source code follows a clear separation of concerns. Quantum state preparation is separated from hardware execution and post-processing. This modular design allows the implementation to work easily across both simulated and real hardware environments.

4.2.1 Quantum State Preparation

The `build_bb84_circuit` function creates an n -qubit quantum circuit based on the classically generated input arrays.

Alice's Encoding: Alice prepares qubits using a combination of X and H gates. This maps her classical bits to the four BB84 quantum states: $|0\rangle$, $|1\rangle$, $|+\rangle$, and $|-\rangle$.

```
for i in range(n):
    if alice_bases[i] == 'Z':
        if alice_bits[i] == 1:
            qc.x(qr[i])
    else: # 'X' basis
        if alice_bits[i] == 1:
            qc.x(qr[i])
        qc.h(qr[i])
```

Eavesdropping Model: When enabled, Eve's interception is modeled by inserting mid-circuit measurements into a dedicated classical register (`cr_eve`). Following the measurement, Eve re-prepares the state according to her chosen basis. This accurately simulates the physical disturbance introduced by a man-in-the-middle attack.

```
if eve_bases is not None:
    for i in range(n):
        if eve_bases[i] == 'X':
            qc.h(qr[i])
        # Eve measures into her specific classical register
        qc.measure(qr[i], cr_eve[i])
        # Eve re-prepares the state based on her measurement
        if eve_bases[i] == 'X':
            qc.h(qr[i])
        # no else branch since Z basis does nothing
```

Bob’s Measurement: Bob performs measurements according to his independently chosen bases. For measurements in the diagonal (X) basis, an H gate is applied to rotate the state back before final measurement in the computational (Z) basis.

```
# 3. Bob's measurements
for i in range(n):
    if bob_bases[i] == 'X':
        qc.h(qr[i])

    # Measure in computational (Z) basis
    qc.measure(qr[i], cr_bob[i])
```

4.2.2 Classical Post-Processing

The function `sift_key` compares the bases chosen by Alice and Bob and removes all positions where the bases do not match. This produces the sifted key. Subsequently, the `compute_qber` function evaluates the reliability of the channel by randomly selecting a subset of the sifted key (by default, 50%) as check bits. These bits are publicly compared to estimate the error rate. The remaining, unrevealed bits form the final secret key.

4.2.3 Execution Architecture

The implementation uses two different execution paths to handle the requirements of theoretical simulation versus physical hardware deployment.

Single-shot execution (`shots = 1`): Standard quantum algorithms (such as VQE or QAOA) typically run the same circuit many times to estimate probability distributions and mitigate noise via majority voting. However, this approach critically fails for QKD. BB84 models a one-time physical transmission where each qubit corresponds to a single, indivisible photon. Executing a circuit multiple times with `shots > 1` incorrectly models the repeated transmission of identical quantum states, violating the No-Cloning Theorem and artificially suppressing the disturbance caused by eavesdropping. Therefore, the architecture strictly enforces `shots = 1` to preserve cryptographic validity.

Hardware batching for scalability: Because `shots=1` is required, generating large key sizes creates practical limitations on real hardware. IBM Quantum hardware imposes a limit on the number of qubits per circuit for example 156 qubits for `ibm_marrakesh`. To resolve this, the hardware orchestrator (`run_bb84_hardware_batch`) splits the total required key length into a batch of smaller, independent circuits such as ten independent 100-qubit circuits. Each circuit is compiled with freshly randomized bases and unique seeds, preventing basis-reuse vulnerabilities. This batch is submitted as a unified array to the Qiskit Runtime `SamplerV2` primitive, incurring only a single queue delay. The results

are concatenated post-execution, allowing the system to scale securely without exceeding physical topology limits.

4.3 Execution Strategy

The main goal of the simulation is to verify that the protocol can generate a secure key under realistic conditions and reliably detect adversarial interference by monitoring the Quantum Bit Error Rate (QBER). Following the foundational security proof by Shor and Preskill [13], a secure secret key can be extracted from the BB84 protocol if and only if the QBER satisfies $e < 11\%$.

To evaluate both functionality and robustness against this threshold, the experiments are designed to isolate variables by testing identical parameters across three distinct environments:

- **Ideal Simulator:** Provides a mathematical baseline with no noise, where the expected QBER is exactly 0.0%.
- **Noisy Simulator:** Introduces parameterized depolarising (1%) and readout (2%) errors to observe natural variations in QBER with and without any attack.
- **Physical Hardware:** Executes batched circuits on IBM Quantum devices to evaluate real-world performance under hardware noise and physical decoherence limitations.

4.3.1 Testing Framework and Results

Five test cases are used to demonstrate protocol correctness across the defined execution environments:

1. **Noiseless Baseline:** With $n = 1000$, `shots = 1`, and no eavesdropper, the noiseless simulator produces $\text{QBER} = 0.00\%$ and a sifted key length of 504 bits. This confirms that Alice’s encoding, the quantum channel, Bob’s measurement, and the sifting logic are mathematically flawless.
2. **Eavesdropper Detection:** In this scenario, Eve intercepts and measures every transmitted qubit. As the key length n increases, the observed QBER approaches the theoretical value of approximately 25%. For key lengths of $n = 1000$ the QBER is 23.02%. This behavior indicates that the protocol can reliably detect the presence of an eavesdropper and trigger an abort when necessary.
3. **Noisy Backend Resilience:** Under simulated hardware conditions (1% depolarising, 2% readout error), an uncompromised channel yields a QBER of 2.8%, remaining safely below the 11% security threshold. When Eve is introduced to the noisy channel, the QBER rises to 27.4%, demonstrating a clear, detectable separation between natural hardware noise and adversarial interference.

4. **Statistical Scalability:** The ideal simulator is evaluated across scaling key lengths $n \in \{50, 100, 500, 1000, 2000\}$. Without Eve, QBER remains 0.00% regardless of scale. With Eve, the QBER smoothly converges to 25%, and the sifted key retention rate stabilises at 50%, satisfying the theoretical expectations of uniform basis sampling governed by the Law of Large Numbers.
5. **Real IBM Hardware Deployment:** Ten independent 100-qubit circuits ($N = 1000$) are batched and executed on the `ibm_marrakesh` processor. In the absence of an eavesdropper, the natural physical noise yields a baseline QBER of 1.2%. When an active attack is simulated on the physical chip, the QBER immediately spikes to 25%. This shows that the intercept-resend attack remains highly visible and securely detectable even when subjected to real-world hardware noise.

Chapter 5

Discussion and Conclusion

5.1 Discussion of Simulation and Hardware Results

5.1.1 Ideal Simulation

The noiseless `AerSimulator` produces $\text{QBER} = 0.00\%$ in the absence of an eavesdropper for all tested key lengths. This matches the theoretical expectation: without noise, Alice and Bob obtain identical bits whenever their bases match, resulting in zero disagreements. This confirms the correctness of circuit construction, basis encoding, measurement, and sifting

Under an intercept-resend attack, the QBER increases to 36.36% at $n = 100$, 30.61% at $n = 200$, and converges to 26.98% at $n = 1000$. The higher variance at lower qubit volumes is a direct consequence of finite-key statistical effects. Because roughly 50% of transmitted bits are discarded during sifting, and half of the remainder are sacrificed for error estimation, small n values produce highly volatile error estimators. As n scales, QBER reliably converges toward the theoretical 25% expectation. Crucially, all adversarial test cases strictly exceeded the 11% abort threshold, ensuring the protocol consistently detects the attack.

5.1.2 Noisy Simulation

To evaluate the protocol's resilience, a parameterised noise model (1% depolarising error per gate and 2% readout error) was introduced. At $n = 1000$, the secure channel achieved a baseline of $\text{QBER} = 2.8\%$. This metric accurately reflects the accumulation of gate

imperfections and measurement noise while remaining safely below the 11% threshold. It shows the protocol will not trigger false-positive aborts under moderate noise. When Eve was introduced to the noisy channel, the QBER spiked to 27.4%. The increase to 27.4% under attack demonstrates a clear separation between natural noise and adversarial disturbance.

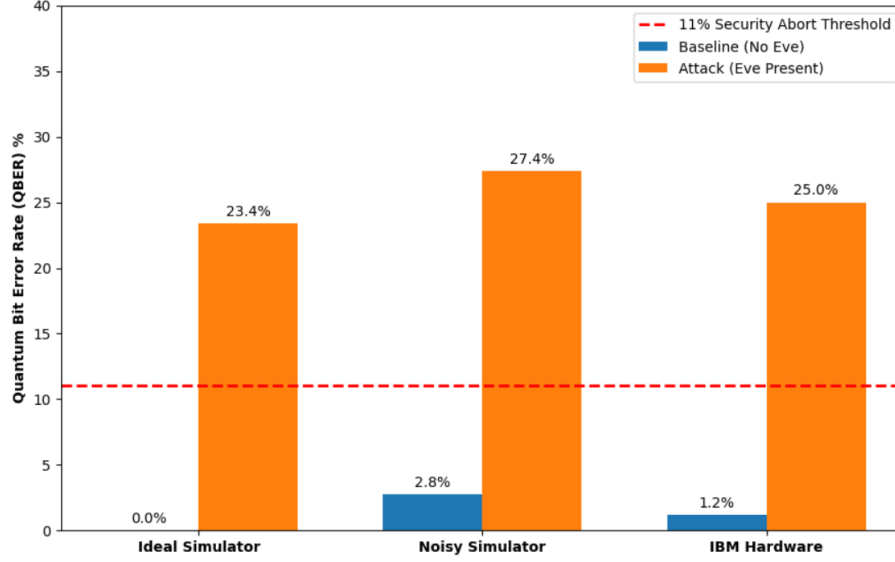


Figure 5.1: QBER comparison across ideal, noisy, and physical hardware environments, illustrating the separation between baseline noise and adversarial disturbance.

5.1.3 Hardware Execution and Environmental Comparison

The hardware experiment was conducted using ten independent 100-qubit circuits ($n = 1000$ total), submitted as a single job to `ibm_marrakesh`. As illustrated in Figure 5.1, the protocol effectively distinguishes between physical device noise and an active eavesdropper.

Without Eve, the physical hardware recorded a baseline QBER of 1.2%, which is slightly higher than the ideal simulator result but still below the selected (11%) threshold. With Eve present, the QBER increased to 25.0%, exceeding the hardware baseline by 23.81 percentage points. This value significantly exceeds the 11% threshold, confirming that intercept-resend attacks remain clearly detectable even under real device noise. The implementation therefore, achieves its intended functionality on physical quantum hardware.

5.1.4 Statistical Convergence

Figure 5.2 shows the convergence of key protocol metrics as n increases from 50 to 2000. The sifted key retention rate rises from 34% at $n = 50$ to 50.6% at $n = 2000$, approaching

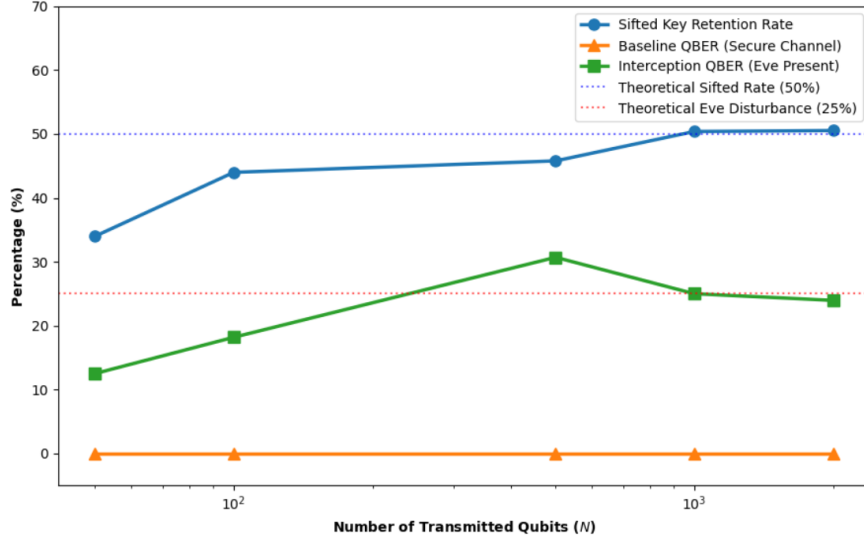


Figure 5.2: Statistical convergence of key retention and error rates as the number of transmitted qubits (n) scales from 50 to 2000.

the theoretical 50% limit predicted by uniform random basis selection.

The baseline QBER securely remains at 0.00% across all values of n , indicating that expanding the circuit depth does not inject artificial software errors. In contrast, the Eve-present QBER converges from 12.50% at $n = 50$ to 23.96% at $n = 2000$, approaching the expected 25%. The $n = 50$ outlier exposes a critical vulnerability: at very small key lengths, statistical variance can accidentally suppress Eve’s error rate dangerously close to the 11.04% threshold. However, for all larger n , the QBER exceeds the threshold, guaranteeing reliable detection.

5.1.5 Limitations

Two main limitations apply. First, Eve is modelled as intercepting every qubit; partial attacks would yield proportionally lower QBER values. Second, very small key sizes like $n = 50$ provide unreliable QBER estimates, highlighting the importance of large sample sizes in practical QKD systems.

5.2 Conclusion

This project successfully implemented the basic BB84 Quantum Key Distribution protocol in Qiskit and evaluated it across three execution environments: noiseless simulation, noisy simulation, and real IBM Quantum hardware.

The ideal simulation produced $\text{QBER} = 0.00\%$ in the absence of eavesdropping, confirming that the core protocol logic is correct. Eve’s intercept-resend attack consistently

raised the QBER above the 11% security threshold across all tested key lengths, with the QBER converging toward the theoretical 25% as n increased. This confirms that BB84's disturbance-based detection mechanism is correctly implemented.

The real hardware results on `ibm_marrakesh` also support this conclusion. The no-Eve hardware run had a small nonzero QBER due to device noise, while the Eve case produced a much higher QBER above the threshold. Therefore, the attack would be detected and the key would be rejected.

Following the security proofs of Shor and Preskill [13], the secret key rate fraction R is given by

$$R = 1 - 2h_2(e),$$

where $h_2(e)$ is the binary entropy function and e is the QBER. A positive key rate is achievable only when $e < 11\%$.

In the noiseless, no-eavesdropper case, we have $e = 0.00\%$, $R = 1.00$, corresponding to a perfectly secure channel. On IBM hardware without an eavesdropper, the observed QBER is 1.2%, yielding $R \approx 0.82$. This indicates the channel remains highly secure and capable of generating usable cryptographic keys despite minor physical decoherence. However, when an eavesdropper was introduced to the hardware, the QBER spiked to 25%, significantly exceeding the security threshold. This results in a negative key rate ($R < 0$), meaning that no secure key can be extracted. The protocol correctly identifies the channel as compromised.

In conclusion, the project shows that BB84 key generation, basis sifting, QBER estimation, eavesdropper detection, noise characterisation, scalability in simulation, and successful execution on real quantum hardware. Thus, fulfilling all objectives stated in the introduction. The results prove that while Alice and Bob can efficiently extract secure cryptographic keys over noisy quantum channels, the fundamental laws of quantum mechanics strictly forbid an eavesdropper from intercepting that key undetected.

Bibliography

- [1] Nicolas Gisin et al. "Quantum Cryptography". In: *Reviews of Modern Physics* 74.1 (2002), pp. 145–195. DOI: 10.1103/RevModPhys.74.145. URL: <https://arxiv.org/abs/quant-ph/0101098>.
- [2] Charles H. Bennett and Gilles Brassard. *Quantum Cryptography: Public Key Distribution and Coin Tossing*. Proceedings of IEEE International Conference on Computers, Systems and Signal Processing, Bangalore, India, pp. 175–179. 1984. arXiv: 2003.06557 [cs.CR]. URL: <https://arxiv.org/pdf/2003.06557>.
- [3] T. M. Forcer et al. "Superposition, Entanglement and Quantum Computation". In: *Quantum Information and Computation* 2.2 (2002), pp. 97–116. URL: <https://www.rintonpress.com/xqic1/qic-2-2/097-116.pdf>.

- [4] William K. Wootters and Wojciech H. Zurek. “A Single Quantum Cannot Be Cloned”. In: *Nature* 299.5886 (1982), pp. 802–803. DOI: 10.1038/299802a0. URL: <https://www.nature.com/articles/299802a0>.
- [5] SujayKumar Reddy M and Chandra Mohan B. *Comprehensive Analysis of BB84, A Quantum Key Distribution Protocol*. 2023. arXiv: 2312.05609 [quant-ph]. URL: <https://arxiv.org/html/2312.05609v1>.
- [6] Miralem Mehic et al. “Implementation of Quantum Key Distribution Network Simulation Module in the Network Simulator NS-3”. In: *Quantum Information Processing* 16.253 (2017). DOI: 10.1007/s11128-017-1702-z. URL: <https://doi.org/10.1007/s11128-017-1702-z>.
- [7] European Telecommunications Standards Institute. *Quantum Key Distribution QKD; Application Interface*. Tech. rep. ETSI GS QKD 004 V2.1.1. ETSI, 2020. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/004/02.01.01_60/gs_qkd004v020101p.pdf.
- [8] European Telecommunications Standards Institute. *Quantum Key Distribution QKD; Protocol and Data Format of REST-Based Key Delivery API*. Tech. rep. ETSI GS QKD 014 V1.1.1. ETSI, 2019. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01.01.01_60/gs_qkd014v010101p.pdf.
- [9] Abdulbast A. Abushgra. “Variations of QKD Protocols Based on Conventional System Measurements: A Literature Review”. In: *Cryptography* 6.1 (2022), p. 12. DOI: 10.3390/cryptography6010012. URL: <https://www.mdpi.com/2410-387X/6/1/12>.
- [10] Xiang-Bin Wang. “Beating the Photon-Number-Splitting Attack in Practical Quantum Cryptography”. In: *Physical Review Letters* 94.23 (2005), p. 230503. DOI: 10.1103/PhysRevLett.94.230503. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.94.230503>.
- [11] Qiskit. *Qiskit Documentation*. Accessed: 2026-05-31. 2026. URL: <https://qiskit.org/docs>.
- [12] IBM Quantum. *IBM Quantum Documentation*. Accessed: 2026-05-31. 2026. URL: <https://quantum.cloud.ibm.com/docs>.
- [13] Peter W. Shor and John Preskill. “Simple Proof of Security of the BB84 Quantum Key Distribution Protocol”. In: *Physical Review Letters* 85.2 (2000), pp. 441–444. ISSN: 1079-7114. DOI: 10.1103/physrevlett.85.441. URL: <http://dx.doi.org/10.1103/PhysRevLett.85.441>.